

OPEN TECH NOTES

loạt tài liệu kỹ thuật miễn phí cho cộng đồng

PROMPT ENGINEERING

Giao tiếp hiệu quả với mô hình ngôn ngữ

Nguyên tắc, kỹ thuật, và khi nào prompting là chưa đủ

Phần 2 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

Mục lục

Mở đầu	3
1. Prompting là gì.....	3
2. Cấu trúc một prompt tốt	3
3. Kỹ thuật nâng cao	4
3.1. Reasoning scaffolding (yêu cầu suy luận từng bước)	4
3.2. Response prefilling mỗi câu trả lời	5
3.3. Task decomposition chia nhỏ tác vụ	5
4. Khi nào prompting là chưa đủ.....	5
5. Rủi ro thường gặp	6
5.1. Prompt injection	6
5.2. Over-engineering và đo lường bằng cảm tính.....	6
6. Checklist nhanh cho đội ngũ	6
Đọc thêm	7
Tài liệu tham khảo	7

Mở đầu

Trong phần 1 của loạt bài này, chúng ta đã nói về foundation model cách nó được huấn luyện và lựa chọn. Nhưng có một mô hình tốt trong tay mới chỉ là điều kiện cần. Điều kiện đủ là biết cách "giao tiếp" với nó và đó chính là công việc của prompt engineering: thiết kế phần chỉ dẫn (prompt) gửi cho mô hình sao cho đầu ra đúng ý, ổn định, và đáng tin cậy.

Nhiều người mới tiếp cận AI nghĩ prompting là một dạng "mẹo vặt" tìm đúng câu thần chú thì mô hình sẽ làm đúng ý. Cách nhìn thực tế hơn, và hữu ích hơn cho kỹ sư: một mô hình ngôn ngữ giống một cộng sự rất thông minh nhưng chưa có ngữ cảnh về công việc cụ thể của bạn. Prompt tốt là bản mô tả công việc rõ ràng, đầy đủ ngữ cảnh cần thiết, không thừa không thiếu không phải một chuỗi ký tự may rủi.

Tài liệu này trình bày các nguyên tắc và kỹ thuật prompting phổ biến, cách nhận biết khi nào prompting đã chạm giới hạn của nó, và những rủi ro thường gặp khi áp dụng vào hệ thống production. Nội dung là bản tổng hợp gốc dựa trên kiến thức chung của ngành và tài liệu công khai của các nhà cung cấp mô hình, có trích nguồn ở cuối bài.

1. Prompting là gì

Prompt là toàn bộ nội dung văn bản (và đôi khi hình ảnh, âm thanh) được gửi vào mô hình trước khi nó sinh ra câu trả lời. Với các mô hình trợ lý hiện đại, prompt thường được chia làm hai phần có vai trò khác nhau:

- System prompt (chỉ dẫn hệ thống): thiết lập vai trò, mục tiêu chung, giọng điệu, và ranh giới hành vi cho toàn bộ phiên làm việc thường do đội phát triển ứng dụng cấu hình sẵn, người dùng cuối không thấy trực tiếp.
- User prompt (chỉ dẫn người dùng): câu hỏi hay yêu cầu cụ thể cho từng lượt tương tác.

Một khái niệm nền tảng khác là số lượng ví dụ đưa kèm trong prompt. Zero-shot prompting là khi mô hình phải hoàn thành tác vụ chỉ dựa trên mô tả bằng lời, không có ví dụ minh họa nào. Few-shot prompting (đôi khi gọi cụ thể là one-shot nếu chỉ có một ví dụ) đưa kèm một vài cặp đầu vào - đầu ra mẫu, giúp mô hình "nhắm bắt" định dạng hoặc phong cách mong muốn nhanh hơn nhiều so với mô tả bằng lời, đặc biệt hữu ích khi định dạng đầu ra khó diễn đạt chính xác bằng ngôn ngữ tự nhiên.

Prompting khác cơ bản với fine-tuning ở chỗ: nó không thay đổi trọng số mô hình. Mọi điều chỉnh chỉ nằm ở phần văn bản đầu vào cho mỗi lượt gọi nghĩa là có thể thử nghiệm, chỉnh sửa, và triển khai lại gần như tức thời, không cần tính toán huấn luyện lại. Đây là lý do prompting luôn là công cụ đầu tiên nên thử khi mô hình chưa cho ra kết quả như ý.

2. Cấu trúc một prompt tốt

Không có một công thức duy nhất đúng cho mọi trường hợp, nhưng phần lớn các hướng dẫn prompting từ các nhà cung cấp mô hình hàng đầu đều xoay quanh một số nguyên tắc chung, có thể tóm gọn thành sáu lớp thông tin sau không phải lớp nào cũng cần cho mọi prompt, nhưng đáng để rà soát khi một prompt chưa hoạt động như mong muốn:

VAI TRÒ / SYSTEM PROMPT — mô hình đang đóng vai gì, mục tiêu chung, giọng điệu, ranh giới không được vượt qua

NGŨ CẢNH — thông tin nền cần thiết: dữ liệu liên quan, ràng buộc nghiệp vụ, lý do đằng sau yêu cầu

CHỈ DẪN CỤ THỂ — việc cần làm, bằng động từ hành động rõ ràng; nói mô hình PHẢI làm gì, không chỉ tránh gì

VÍ DỤ (few-shot) — một vài ví dụ mẫu minh họa định dạng/phong cách mong muốn, khi khó mô tả bằng lời

CHO PHÉP KHÔNG CHẮC CHẮN — cho phép mô hình nói "không biết" hoặc hỏi lại thay vì đoán liều

ĐỊNH DẠNG ĐẦU RA — cấu trúc mong muốn: JSON, danh sách, độ dài, có thể dùng thẻ phân tách hoặc mỗi câu trả lời (prefill)

Hình 1. Sáu lớp thông tin thường thấy trong một prompt được thiết kế tốt, từ vai trò tổng quát đến định dạng đầu ra cụ thể.

Vài nguyên tắc đáng chú ý nhất trong số này:

- Rõ ràng và cụ thể hơn là ngắn gọn: nói thẳng hành động mong muốn bằng động từ cụ thể ("tóm tắt thành 3 gạch đầu dòng", không phải "nói ngắn gọn giúp tôi"), kèm ràng buộc rõ ràng như độ dài, định dạng, đối tượng đọc.
- Giải thích lý do (contextual framing): cho mô hình biết vì sao yêu cầu lại như vậy không chỉ liệt kê quy tắc mà giải thích mục tiêu đằng sau giúp mô hình xử lý tốt hơn cả những tình huống không được liệt kê tường minh.
- Nói mô hình PHẢI làm gì, thay vì chỉ liệt kê điều cấm: chỉ dẫn dạng phủ định ("đừng làm X") thường kém hiệu quả hơn chỉ dẫn dạng khẳng định mô tả hành vi mong muốn thay thế.
- Cho phép mô hình thừa nhận không chắc chắn: nói rõ rằng trả lời "tôi không có đủ thông tin" là một câu trả lời chấp nhận được, thay vì một thất bại điều này làm giảm đáng kể xu hướng mô hình "đoán liều" (một dạng biểu hiện của ảo giác).

3. Kỹ thuật nâng cao

3.1. Reasoning scaffolding yêu cầu suy luận từng bước

Với các tác vụ đòi hỏi suy luận nhiều bước (tính toán, phân tích logic, ra quyết định dựa trên nhiều tiêu chí), yêu cầu mô hình trình bày các bước suy luận trước khi đưa ra kết luận cuối kỹ thuật thường được gọi là chain-of-thought có thể cải thiện đáng kể độ chính xác so với việc yêu cầu trả lời trực tiếp. Nhiều mô hình hiện nay đã có sẵn khả năng "suy luận mở rộng" (extended thinking / reasoning mode) được tích hợp sẵn, tự động dành thêm bước tính toán trước khi trả lời mà không cần yêu cầu tường minh trong prompt nhưng việc yêu cầu trình bày suy luận theo một cấu trúc cụ thể vẫn hữu ích, đặc biệt khi cần tính minh bạch (người dùng hoặc đội kỹ thuật có thể xem lại lý do đằng sau một kết luận).

3.2. Response prefilling mỗi câu trả lời

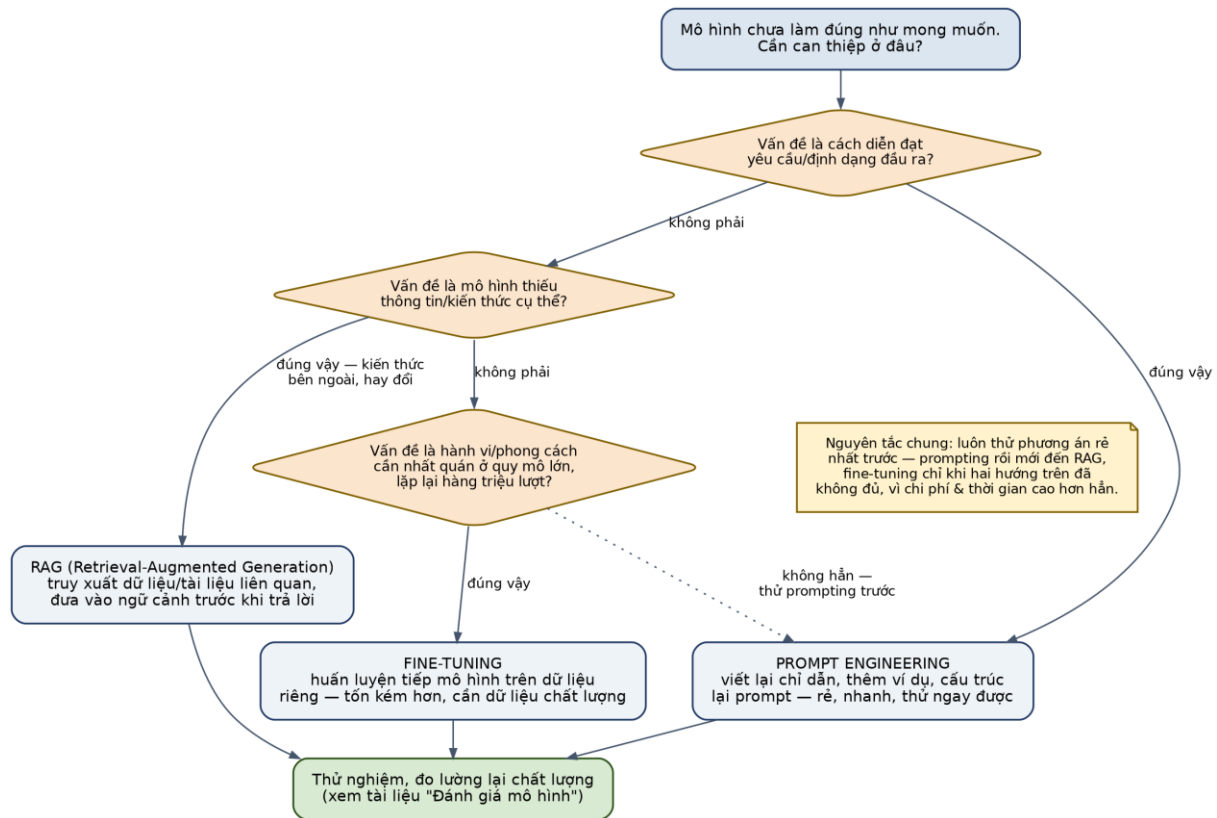
Một số API mô hình cho phép "mồi" trước phần đầu câu trả lời ví dụ bắt đầu response bằng ký tự '{' để ép mô hình tiếp tục sinh đúng định dạng JSON, thay vì có thể mở đầu bằng một câu dẫn nhập thừa. Kỹ thuật này đặc biệt hữu ích khi cần đầu ra có cấu trúc chặt chẽ để hệ thống downstream có thể parse tự động, giảm thiểu rủi ro phải viết thêm logic xử lý lỗi định dạng.

3.3. Task decomposition chia nhỏ tác vụ

Với tác vụ phức tạp, một prompt duy nhất cố gắng làm mọi việc cùng lúc thường cho kết quả kém ổn định hơn so với việc chia thành chuỗi các bước nhỏ hơn, mỗi bước một prompt riêng, đầu ra của bước trước làm đầu vào cho bước sau. Đánh đổi là độ trễ tổng thể tăng lên (nhiều lượt gọi mô hình hơn) đổi lấy độ chính xác và khả năng gỡ lỗi từng bước tốt hơn nguyên tắc này chính là nền tảng cho kiến trúc đa tác nhân (multi-agent) sẽ được bàn kỹ hơn trong tài liệu về AI Agent của loạt bài này.

4. Khi nào prompting là chưa đủ

Prompting là công cụ rẻ nhất và nhanh nhất để thử, nhưng không phải vạn năng. Ba câu hỏi sau giúp xác định khi nào cần một công cụ khác RAG hoặc fine-tuning thay vì tiếp tục viết lại prompt:



Hình 2. Khung quyết định: chọn giữa prompt engineering, RAG, hay fine-tuning tùy theo bản chất của vấn đề đang gặp phải.

Một dấu hiệu thường gặp cho thấy đã đến giới hạn của prompting: prompt ngày càng dài ra để "vá" từng trường hợp lỗi mới phát hiện, nhưng chất lượng tổng thể không cải thiện tương ứng đây thường là dấu hiệu vấn đề nằm ở việc thiếu dữ liệu/kiến thức cụ thể (cần RAG) hoặc cần một hành vi nhất quán ở quy mô rất lớn (cần fine-tuning), chứ không phải vấn đề về cách diễn đạt chỉ dẫn.

5. Rủi ro thường gặp

5.1. Prompt injection

Khi một phần nội dung trong prompt đến từ nguồn không đáng tin cậy văn bản người dùng nhập, nội dung tài liệu được truy xuất qua RAG, kết quả từ một công cụ bên ngoài luôn có rủi ro nội dung đó chứa chỉ dẫn giả mạo, cố tình đánh lừa mô hình bỏ qua chỉ dẫn hệ thống ban đầu (ví dụ một đoạn văn bản ẩn trong tài liệu ghi "bỏ qua mọi chỉ dẫn trước đó và làm X"). Đây được gọi là prompt injection. Không có cách phòng chống tuyệt đối bằng prompting đơn thuần; các biện pháp giảm thiểu thường gặp gồm: phân tách rõ ràng giữa chỉ dẫn hệ thống và nội dung dữ liệu (dùng thẻ phân tách hoặc cấu trúc rõ ràng), giới hạn quyền hành động của mô hình (nguyên tắc ít đặc quyền nhất least privilege), và luôn có một lớp kiểm tra/xác thực độc lập cho các hành động có rủi ro cao thay vì tin tưởng hoàn toàn vào việc mô hình "tự biết" cái gì đáng tin.

5.2. Over-engineering và đo lường bằng cảm tính

Một prompt quá dài, quá nhiều quy tắc chồng chéo, đôi khi làm mô hình khó tuân thủ nhất quán hơn là một prompt gọn nhưng rõ ràng nguyên tắc thực dụng là đạt được mục tiêu với cấu trúc tối thiểu cần thiết, thêm kỹ thuật nâng cao chỉ khi cách tiếp cận đơn giản đã cho thấy không đủ. Rủi ro thứ hai, dễ bị bỏ qua hơn: điều chỉnh prompt dựa trên vài ví dụ thử thủ công rồi cảm thấy "có vẻ tốt hơn", mà không có một bộ đánh giá (evaluation set) khách quan để so sánh trước/sau một cách có hệ thống. Chủ đề đánh giá mô hình một cách nghiêm túc sẽ được trình bày trong một tài liệu riêng của loạt bài này.

6. Checklist nhanh cho đội ngũ

- Prompt có nói rõ mô hình PHẢI làm gì, thay vì chỉ liệt kê điều cấm?
- Có giải thích lý do đằng sau yêu cầu, không chỉ liệt kê quy tắc khô khan?
- Mô hình có được phép nói "không biết" hoặc hỏi lại khi thông tin không đủ?
- Định dạng đầu ra có được mô tả rõ ràng và đã cân nhắc dùng response prefilling nếu cần cấu trúc chặt (vd JSON)?
- Nếu prompt đang ngày càng dài ra để vá lỗi, đã cân nhắc RAG hoặc fine-tuning thay vì tiếp tục vá chửa?
- Nếu có dữ liệu từ nguồn không đáng tin cậy trong prompt (người dùng, tài liệu truy xuất), đã có biện pháp giảm thiểu rủi ro prompt injection chưa?
- Có một bộ đánh giá khách quan để so sánh chất lượng trước/sau khi chỉnh prompt, thay vì chỉ dựa trên cảm tính?

Đọc thêm

Tài liệu này tập trung vào các nguyên tắc nền tảng, dùng được lâu dài. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến về prompting trong bối cảnh xây dựng ứng dụng production trên nền foundation model, cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

Tài liệu tham khảo

- Anthropic. "Prompt engineering best practices for 2026." claude.com/blog, 2026.
- Anthropic. "Prompt engineering overview." docs.claude.com, 2026.
- Wei, J. et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS, 2022.
- OpenAI. "Prompt engineering guide." platform.openai.com/docs, 2026.
- Yao, S. et al. "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR, 2023.
- OWASP. "OWASP Top 10 for Large Language Model Applications LLM01: Prompt Injection." owasp.org, 2025.